Konzeption und Evaluation eines domänenbasierten Explorers als VS-Code Extension für SAP CAP Anwendungen

Bachelorarbeit

aus dem Studiengang Wirtschaftsinformatik Software Engineering

an der Dualen Hochschule Baden-Württemberg Mannheim

von

Max Christian Meinel

15.04.2026

Matrikelnummer, Kurs:	7864687, WWI23SEB
Unternehmen:	SAP SE, 69190, Walldorf
Abteilung:	CAP Tools & MTX
Leiter des Studiengangs:	Prof. Dr. Henning Pagnia
Betreuer im Unternehmen:	Christian Fuchs
Betreuer an der DHBW:	Ulrich Wolf

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden- Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Konzeption und Evaluation eines domänenbasierten Explorers als VS-Code Extension für SAP CAP Anwendungen

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Walldorf 15.04.2026

Max Christian Meinel

Abstract

Abstract :)

Inhaltsverzeichnis

1 Einleitung ... 1
1.1 Motivation ... 1
1.2 Problemstellung ... 1
1.3 Zielsetzung der Arbeit ... 1
1.4 Forschungsfragen ... 1
1.5 Aufbau der Arbeit ... 1

2 Grundlagen ... 2
2.1 SAP Cloud Application Programming Model (CAP) ... 2
2.2 Language Server und Language Server Protocol ... 4
2.3 VS Code Extension Architektur ... 5

3 Methodik ... 7
3.1 Design Science Research als methodischer Rahmen ... 7
3.2 Auswahl des DSR-Prozessmodells ... 8
3.3 Anwendung des DSR-Prozesses in dieser Arbeit ... 9

4 Problem- und Anforderungsanalyse ... 11
4.1 Problemkontext und Analyse von CAP-Repositories ... 11
4.2 Identifizierte Herausforderungen bei der Navigation ... 15
4.3 Ableitung der Designziele ... 17
4.4 Funktionale Anforderungen ... 18
4.5 Abgrenzung der Arbeit ... 19

5 Integrationsstrategien ... 21
5.1 Repository-basierte Integration ... 21
5.2 CSN-basierte Integration ... 22
5.3 Language-Server-basierte Integration ... 23
5.4 Vergleich und Auswahl ... 25
5.5 Auswahl der Integrationsstrategie ... 28

6 Implementierung des Prototyps ... 29
6.1 Architekturkonzept und Designentscheidungen ... 29
6.2 Integration mit CDS Language Server ... 32
6.3 Datenmodell und Tree-Building ... 34
6.4 State Management und Visualisierung ... 34
6.5 Such- und Filtersystem ... 34

6.6 Navigation und Interaktion 34

7 Evaluation 35
7.1 Evaluationskriterien 35
7.2 Vergleich der Integrationsansätze 35
7.3 Analyse der Ergebnisse 35

8 Diskussion und Fazit 36
8.1 Interpretation der Ergebnisse 36
8.2 Limitationen 36
8.3 Zusammenfassung der Ergebnisse 36
8.4 Beantwortung der Forschungsfragen 36
8.5 Ausblick auf zukünftige Arbeiten 36

Literatur ..... a

Abbildungsverzeichnis

Abbildung 1 CAP-Architektur mit CDS als zentralem Baustein [1] 2

Abbildung 2 CDS-Kompilierungsfluss mit CSN als zentralem Zwischenformat [2] 3

Abbildung 3 Konzeptionelle Fragmentierung eines CAP-Services (vereinfacht). 13

Abbildung 4 Navigationskomplexität typischer Entwicklungsaufgaben im Bookshop-Projekt 16

Quellcode 5 CDS-Service-Definition mit Projektionen und Action 21

Quellcode 6 Ausschnitt eines CSN-Modells mit Service und Entity 23

Quellcode 7 Beispiel einer Workspace-Symbol-Antwort des CAP Language Servers 24

Abbildung 8 Ergebnis der Nutzwertanalyse 28

Abbildung 9 Komponentenstruktur und Datenfluss der VS Code Extension 31

Tabellenverzeichnis

Tabelle 1 Zentrale Artefakte eines CAP-Services (eigene Darstellung basierend auf [3, Sec. Domain Modeling]) 14

Tabelle 2 Bewertungskriterien und Gewichtung 26

Tabelle 3 Nutzwertanalyse der Integrationsstrategien 27

Tabelle 4 Detaillierte Begründung der Punktvergabe in der Nutzwertanalyse . . . . . e

Tabelle 5 Szenario „Service verstehen“ – 3 Dateien, 10 Navigationsschritte . f

Tabelle 6 Szenario „Feld hinzufügen“ – 6 Dateien, 6 Navigationsschritte . . . f

Tabelle 7 Szenario „Auth anpassen“ – 3 Dateien, 4 Navigationsschritte . . . . . f

Tabelle 8 Szenario „Handler debuggen“ – 3 Dateien, 4 Navigationsschritte . g

Codeverzeichnis

Abbildung 1 **Exemplarische Struktur eines SAP CAP-Projekts (vereinfacht) [4].** **12**

Abbildung 2 **Verteilung der CatalogService-Artefakte im Bookshop-Projekt** **17**

Abkürzungsverzeichnis

API	Application Programming Interface
CAP	Cloud Application Programming Model
CDL	CDS Definition Language
CDS	Core Data Services
CQL	Core Query Language
CSN	Core Schema Notation
CXL	Core Expression Language
DSR	Design Science Research
HANA	High-Performance Analytic Appliance
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
LS	Language Server
LSP	Language Server Protocol
RPC	Remote Procedure Call
SAP	Systems, Applications, and Products
SDK	Software Development Kit
UI	User Interface
URI	Uniform Resource Identifier
VS	Visual Studio

1 Einleitung

1.1 Motivation

1.2 Problemstellung

1.3 Zielsetzung der Arbeit

1.4 Forschungsfragen

1.5 Aufbau der Arbeit

2 Grundlagen

2.1 SAP Cloud Application Programming Model (CAP)

Das SAP Cloud Application Programming Model (CAP) ist ein Framework für Enterprise-Cloud-Anwendungen von SAP [5]. Das zentrale Ziel ist die Maximierung der Produktivität durch integrierte Best Practices. Entwickler erhalten bewährte Lösungsmuster standardmäßig bereitgestellt, ohne diese manuell implementieren zu müssen.

Das Framework basiert auf drei Kernprinzipien [5]. Domain-First bedeutet, dass der Fokus auf der Geschäftslogik liegt, während technische Details in den Hintergrund treten. Separation of Concerns strukturiert die Anwendung durch klare Trennung von Domänenmodell (Datenbank), Services und Benutzeroberfläche [6]. Serviceorientierte Architektur bedeutet, dass Funktionalität über klar definierte APIs zugänglich gemacht wird [7].

Abbildung 1 zeigt die Architektur von CAP im Überblick. Core Data Services (CDS) bilden den zentralen Baustein, der Services, Datenmodelle und User Interfaces (UIs) verbindet. CAP unterstützt zwei Laufzeitumgebungen: Node.js und Java.

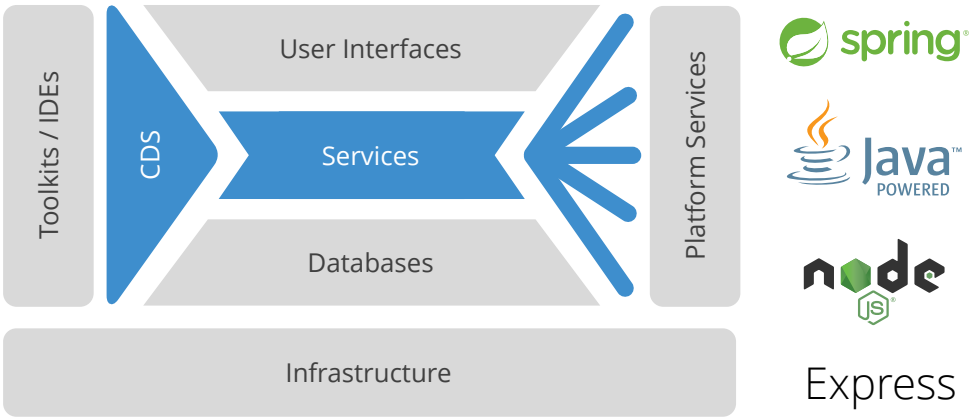

Abbildung 1 — CAP-Architektur mit CDS als zentralem Baustein [1]

Die technischen Features umfassen vollständige Software Development Kits (SDKs) für Node.js und Java als Laufzeitumgebungen. Event Handler werden als JavaScript-, TypeScript-Funktionen oder als Java-Klassen implementiert. CAP unterstützt mehrere Kommunikationsprotokolle wie OData [8], OpenAPI [9] und AsyncAPI, die automatisch aus CDS-Modellen generiert werden. Weitere zentrale Features sind Multi-Tenancy-Unterstützung zur Verwaltung mehrerer Mandanten, Datenbank-Abstraktion für HANA, SQLite und PostgreSQL sowie Event-Driven Messaging für asynchrone Kommunikation.

2.1.1 Core Data Services (CDS)

CDS ist die Modellierungssprache von SAP CAP [2]. Entwickler definieren in CDS die Struktur ihrer Anwendung. Entities repräsentieren Datenmodelle, Services definieren API-Schnittstellen und Associations beschreiben Beziehungen zwischen Entities. Die textuelle Syntax heißt CDS Definition Language (CDL) und wird in Dateien mit der Endung `.cds` geschrieben [10, Sec. Language Preliminaries].

Das CDS-Toolkit kompiliert alle `.cds`-Dateien eines Projekts zu Core Schema Notation (CSN), der kompilierten Repräsentation des Gesamtmodells [11, Sec. Anatomy]. Abbildung 2 zeigt diesen Kompilierungsprozess. CSN repräsentiert das Modell als Datenstruktur und enthält die vollständige semantische Information aller Definitionen.

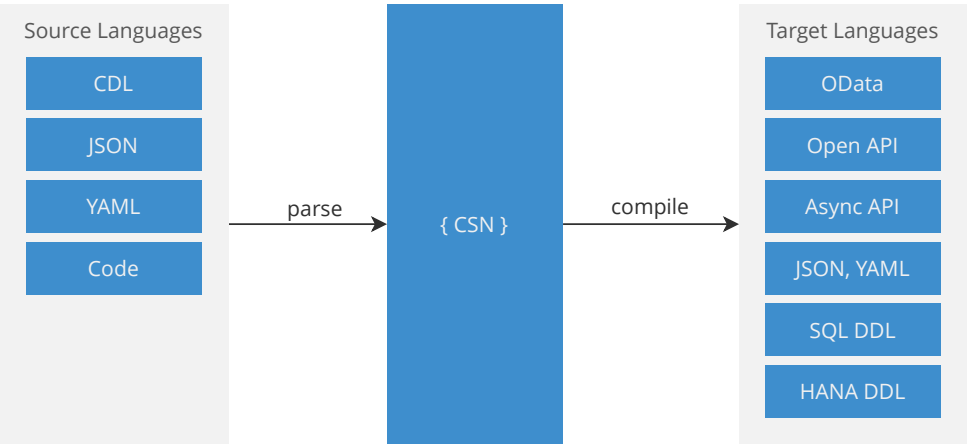

Abbildung 2 — CDS-Kompilierungsfluss mit CSN als zentralem Zwischenformat [2]

Neben CDL und CSN umfasst CDS weitere Notationen wie Core Query Language (CQL) für Abfragen und Core Expression Language (CXL) für Ausdrücke [2]. Diese sind für die vorliegende Arbeit nicht relevant.

2.2 Language Server und Language Server Protocol

Moderne Integrated Development Environments (IDEs) bieten intelligente Funktionen wie Autovervollständigung, Fehleranzeige, Go-to-Definition und Refactoring. Diese Funktionen erfordern ein tiefes Verständnis der Programmiersprache, insbesondere von Syntax, Semantik und Typsystem. Ein Language Server ist ein separater Prozess, der diese Sprachanalyse übernimmt und die entsprechenden IDE-Features bereitstellt.

Das Language Server Protocol (LSP) ist ein von Microsoft entwickeltes Standardprotokoll zur Kommunikation zwischen IDE und Language Server [12]. Das Ziel ist die Entkopplung von Editor und Sprachanalyse. Ein Language Server kann dadurch in allen IDEs genutzt werden. Vor LSP musste jede IDE die Sprachunterstützung selbst implementieren, was zu N×M Implementierungen führte (N IDEs multipliziert mit M Sprachen). Mit LSP genügt es, den Language Server einmal zu implementieren, wodurch nur noch N+M Implementierungen erforderlich sind. Die Kommunikation erfolgt über JavaScript Object Notation Remote Procedure Call (JSON-RPC) [13] mit Request-Response und Notifications.

SAP stellt für CDS einen eigenen Language Server bereit, der die LSP-Funktionalitäten für CDS-Dateien implementiert.

2.2.1 Architektur des Language Servers

LSP basiert auf einem Client-Server-Modell. Die IDE ist der Client, der Language Server ist ein separater Prozess, der als Server fungiert [14, Sec. Base Protocol]. Der Server wird vom Client gestartet, verarbeitet während der Laufzeit Anfragen und wird am Ende wieder beendet. Der Server ist zustandsbehaftet und kennt den aktuellen Stand aller geöffneten Dokumente.

2.2.2 Relevante LSP-Funktionalitäten

LSP definiert verschiedene Funktionen für die Interaktion zwischen IDE und Language Server [14, Sec. Language Features]. Der Client sendet Requests an den Server, die jeweils eine Dokumentenposition oder einen Workspace-Kontext enthalten. Der Server analysiert den Quellcode und liefert die angeforderten Informationen als Response zurück. Relevante Funktionen sind:

- `textDocument/completion`: Liefert Vervollständigungsvorschläge für die aktuelle Eingabeposition
- `textDocument/hover`: Liefert Typ- und Dokumentationsinformationen für ein Symbol an einer gegebenen Position
- `textDocument/definition`: Liefert die Quellcode-Position der Definition eines Symbols
- `workspace/symbol`: Liefert alle im Workspace definierten Symbole mit ihren Positionen

2.3 VS Code Extension Architektur

VS Code ist über Extensions erweiterbar [15]. Extensions sind Plug-ins, die zusätzliche Funktionalität zum Editor hinzufügen. Sie laufen in einem separaten Extension Host Prozess und sind damit vom Editor-Kern isoliert. Extensions können UI-Elemente, Commands und Sprachfeatures beitragen.

VS Code stellt eine Extension API bereit, über die Extensions auf Editor-Funktionen zugreifen können [16]. Die API umfasst verschiedene Bereiche wie Dateiverwaltung, UI-Komponenten, Sprach-Features und Debugging-Funktionen. Eine dieser APIs ist die Tree View API.

2.3.1 Tree View API

Die Tree View API ermöglicht das Erstellen eigener Baumansichten in der Sidebar. Diese Views erscheinen neben den eingebauten Views wie Explorer oder Source Control. Die zentralen Konzepte sind `TreeDataProvider` und `TreeItem`. Der

`TreeDataProvider` liefert die Daten für den Baum. Ein `TreeItem` repräsentiert einen einzelnen Knoten [17].

2.3.2 Integration von Language Servern

VS Code bietet native Unterstützung für Language Server über die `vscode-languageclient` Bibliothek. Eine Extension kann einen Language Server als separaten Prozess starten. Der Language Client übernimmt die LSP-Kommunikation automatisch. Neben den Standard-LSP-Features kann eine Extension auch eigene Requests an den Server senden. So können Analyseergebnisse des Language Servers für eigene UI-Komponenten genutzt werden [18].

3 Methodik

Diese Arbeit folgt dem Design Science Research (DSR) Ansatz zur Entwicklung und Evaluation eines technischen Artefakts. Das zu entwickelnde Artefakt ist eine VS-Code Extension, die Entwickler bei der Navigation in SAP CAP-Projekten unterstützen soll. Dieses Kapitel begründet zunächst die Wahl des methodischen Rahmens und beschreibt anschließend die konkrete Anwendung des DSR-Prozesses in dieser Arbeit.

3.1 Design Science Research als methodischer Rahmen

Die Wahl der Forschungsmethode hängt vom Forschungsziel ab. Verhaltenswissenschaftliche Ansätze erklären bestehende Phänomene und entwickeln Theorien. Design Science Research (DSR) hingegen konstruiert und evaluiert neue Artefakte zur Lösung identifizierter Probleme [19, p. 75]. Diese Arbeit entwickelt eine VS-Code Extension als konkretes technisches Artefakt, das Navigationsprobleme in CAP-Projekten löst. DSR ist daher die geeignete Forschungsmethode.

Alternative Forschungsansätze wurden geprüft und als ungeeignet befunden. Action Research ist ein Ansatz, bei dem Forscher aktiv in Veränderungsprozesse innerhalb einer Organisation eingreifen [20, p. 235]. Die Methode durchläuft iterative Zyklen von Diagnose, Planung, Aktion und Evaluation. Sie setzt voraus, dass Forscher und Praktiker eng in einem spezifischen organisationalen Kontext zusammenarbeiten. Für diese Arbeit ist Action Research ungeeignet, da keine organisationale Intervention angestrebt wird, sondern die Entwicklung eines generischen Werkzeugs.

Case Study Research ermöglicht die tiefgehende Analyse von Phänomenen in ihrem natürlichen Kontext. Case Studies sind explorativ, deskriptiv oder explanatorisch und zielen darauf ab, existierende Phänomene zu verstehen [21, p. 135]. Diese Arbeit hingegen entwickelt ein neues Artefakt und evaluiert dessen Eignung. Case Study Research ist daher nicht geeignet.

3.2 Auswahl des DSR-Prozessmodells

Innerhalb von DSR existieren verschiedene Prozessmodelle, die den Forschungsprozess unterschiedlich strukturieren. Diese Modelle unterscheiden sich in Detaillierungsgrad, Phasenstruktur und Anwendbarkeit auf konkrete Forschungsprojekte.

Das konzeptionelle Framework von Hevner et al. bietet eine grundlegende Struktur mit drei Säulen (Environment, Information Systems Research, Knowledge Base) und den Phasen Build und Evaluate [19, pp. 79–80, Fig. 2]. Dieses Framework eignet sich zur grundsätzlichen Einordnung von DSR-Forschung, bietet jedoch keine detaillierte Prozessstruktur für die praktische Durchführung.

Dresch et al. definieren einen 12-schrittigen DSR-Prozess [22, pp. 118–124, Fig. 6.1], der sich in drei methodische Phasen gliedert. Das Modell unterscheidet explizit zwischen abduktiven, deduktiven und induktiven Schlussformen. Diese epistemologische Fundierung ist wissenschaftlich wertvoll, führt jedoch zu einem hohen Detaillierungsgrad, der für eine anwendungsorientierte Bachelorarbeit nicht erforderlich ist.

Peffers et al. schlagen ein 6-Phasen-Prozessmodell vor, das speziell für Information Systems Research entwickelt wurde. Das Modell strukturiert DSR-Forschung in die Phasen Problem Identification and Motivation, Define Objectives of a Solution, Design and Development, Demonstration, Evaluation und Communication [23, pp. 52–54]. Dieses Modell ist ein weitverbreiteter Standard in der Information-Systems-Community und bietet einen guten Kompromiss zwischen Strukturierung und Praktikabilität.

Für diese Arbeit wurde das Prozessmodell von Peffers et al. gewählt. Die Begründung liegt in mehreren Faktoren. Erstens ist Peffers' Prozessmodell ein weitverbreiteter DSR-Ansatz in der Information-Systems-Forschung, was die wissenschaftliche Anerkennung der Methodik stärkt. Zweitens bietet die 6-Phasen-Struktur einen klaren Rahmen, der im Vergleich zu Dresch's 12 Schritten keine unnötige Komplexität einführt. Drittens enthält das Modell eine explizite Demonstration-Phase, die

ideal zur prototypischen Implementierung dieser Arbeit passt, während Hevner's Framework diese Phase nicht explizit ausweist. Viertens entspricht die sequenzielle Struktur dem tatsächlichen Projektverlauf dieser Arbeit besser als abstrakte epistemologische Differenzierungen.

3.3 Anwendung des DSR-Prozesses in dieser Arbeit

Die sechs Phasen des DSR-Prozesses nach Peffers et al. [23, pp. 52–56] strukturieren den Forschungsprozess dieser Arbeit. Die Phasen dienen als methodischer Rahmen, während die Kapitelstruktur der fachlichen Logik des Untersuchungsgegenstands folgt. Die folgende Darstellung zeigt, wie die methodischen Phasen auf die Kapitel dieser Arbeit abgebildet werden.

Phase 1: Problem Identification and Motivation. Kapitel 4 identifiziert das Navigationsproblem in CAP-Projekten durch Analyse typischer Repository-Strukturen und quantifiziert den Navigationsaufwand anhand repräsentativer Entwicklungsszenarien.

Phase 2: Define Objectives of a Solution. Ebenfalls in Kapitel 4 werden aus den identifizierten Herausforderungen lösungsneutrale Designziele abgeleitet und in konkrete funktionale sowie nicht-funktionale Anforderungen überführt.

Phase 3: Design and Development. Kapitel 5 vergleicht drei Integrationsstrategien zur Gewinnung von Service-Strukturinformationen mittels Nutzwertanalyse und wählt die vorteilhafteste aus. Kapitel 6 implementiert die gewählte Strategie als VS-Code Extension.

Phase 4: Demonstration. Kapitel 6 demonstriert die praktische Anwendbarkeit der Extension am Bookshop-Referenzprojekt.

Phase 5: Evaluation. Kapitel 7 evaluiert das Artefakt kriterienbasiert anhand der definierten Anforderungen durch praktische Erprobung an repräsentativen CAP-Projekten.

Phase 6: Communication. Kapitel 8 interpretiert die Ergebnisse, diskutiert Limitationen, fasst die Erkenntnisse zusammen und reflektiert den Beitrag zur Wissensbasis. Diese Arbeit selbst stellt die primäre Kommunikationsform dar.

4 Problem- und Anforderungsanalyse

Dieses Kapitel konkretisiert die im Methodikteil beschriebene Problemidentifikation und Zieldefinition im Kontext von SAP CAP-Projekten. Zunächst wird die strukturelle Organisation typischer CAP-Repositories analysiert und die Verteilung servicebezogener Artefakte untersucht. Darauf aufbauend werden navigationsbezogene Herausforderungen abgeleitet, aus denen Designziele sowie funktionale und nicht-funktionale Anforderungen an die zu entwickelnde VS-Code Extension formuliert werden. Abschließend wird die Arbeit inhaltlich abgegrenzt.

4.1 Problemkontext und Analyse von CAP-Repositories

4.1.1 Typische Struktur von CAP-Projekten

SAP CAP gibt eine strenge Projektstruktur vor, die das Prinzip der Separation of Concerns umsetzt und damit die Wartbarkeit skalierender Projekte unterstützt [24, Sec. Separation of Concerns]. Ein typisches CAP-Projekt folgt der in Abbildung 1 dargestellten Struktur.

```
example-cap-project
├─ db
│  ├─ schema.cds
│  └─ data
│     └─ seed.csv
├─ srv
│  ├─ catalog-service.cds
│  ├─ catalog-service.js
│  ├─ handlers
│  │  └─ validation.js
│  └─ auth
│     └─ roles.cds
├─ app
│  └─ fiori-app
│     └─ manifest.json
├─ test
│  └─ catalog-service.test.js
├─ config
│  └─ cds.env
└─ package.json
```

Abbildung 1 — Exemplarische Struktur eines SAP CAP-Projekts (vereinfacht) [4].

example-cap-project/

Das Rootverzeichnis bildet die Basis des Projekts und bündelt alle fachlichen, technischen und konfigurativen Artefakte nach dem Prinzip der Separation of Concerns.

db/

Dieses Verzeichnis enthält das fachliche Domänenmodell. Hier werden Entities und ihre Beziehungen in CDS definiert, die die Persistenz- und Datenstrukturebene repräsentieren.

srv/

Dieses Verzeichnis beinhaltet Service-Definitionen und Geschäftslogik. Der Service exponiert ausgewählte Teile des Domänenmodells und verknüpft deklarative Modellierung mit imperativer Implementierung.

app/

Dieses Verzeichnis enthält optionale Frontend-Anwendungen, die die definierten Services konsumieren und die Präsentationsebene repräsentieren.

test/
Dieses Verzeichnis beinhaltet Testfälle zur Validierung der Service-Logik und unterstützt damit Qualitätssicherung und Wartbarkeit.

config/
Dieses Verzeichnis enthält umgebungsspezifische Konfigurationen zur Steuerung von Laufzeit- und Deployment-Parametern.

package.json
Diese Datei definiert Projektmetadaten, Abhängigkeiten sowie Skripte für Build, Start und Tests.

4.1.2 Verteilung servicebezogener Artefakte

Ein CAP-Service besteht nicht aus einem einzelnen Artefakt, sondern setzt sich aus mehreren funktionalen Bereichen zusammen, die unterschiedlichen Verantwortlichkeiten zugeordnet sind. Diese Fragmentierung erfolgt entlang technischer und fachlicher Concerns, wobei die Zuordnung zwischen den Artefakten über das Projekt verteilt ist.

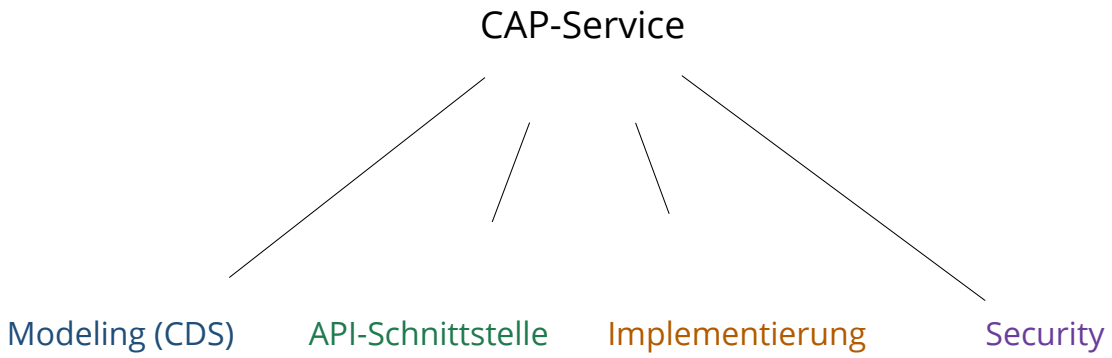

Abbildung 3 — Konzeptionelle Fragmentierung eines CAP-Services (vereinfacht).

Abbildung 3 verdeutlicht, dass ein CAP-Service nicht als monolithisches Artefakt vorliegt, sondern sich über mehrere konzeptionelle Ebenen erstreckt. Modell- und Service-Definition sind konzeptionell getrennt, wobei die Service-Schnittstelle

nicht zwingend das vollständige Domänenmodell abbildet. Das Laufzeitverhalten ist nicht im Modell selbst definiert, sondern in separaten Implementierungen ausgelagert. Sicherheitsrelevante Regeln werden als Annotationen außerhalb der Kernlogik verortet. Dadurch sind fachliche und technische Aspekte strukturell entkoppelt [25, Sec. Focus on Domain].

Kind	**Ebene**	**Beispiele**
Entity	Modeling (CDS)	Books, Authors
Type	Modeling (CDS)	UUID, String, Decimal
Association	Modeling (CDS)	to Author, to Items
Aspect	Modeling (CDS)	Reusable model extensions
Service	API-Schnittstelle	CatalogService
Projection	API-Schnittstelle	Books as projection on ...
Action / Function	API-Schnittstelle	submitOrder(), topSelling()
Event Handler	Implementierung	before / on / after
Authorization Annotations	Security	@restrict, @requires

Tabelle 1 — Zentrale Artefakte eines CAP-Services (eigene Darstellung basierend auf [3, Sec. Domain Modeling])

Tabelle 1 konkretisiert die in der Abbildung 3 dargestellten Ebenen durch die zentralen Artefakt-Kinds des CAP-Modells. Ein vollständiges Serviceverständnis erfordert die Betrachtung mehrerer Artefakt-Kategorien, deren Beziehungen über Referenzen hergestellt werden. Eine zentrale, aggregierte Repräsentation eines Services existiert nicht auf Artefaktebene. Änderungen an einem Service können daher mehrere Ebenen gleichzeitig betreffen, da sich die Service-Identität erst aus dem Zusammenspiel aller Ebenen ergibt. Diese Architektur folgt den Prinzipien von Domain-Driven Design und Separation of Concerns [3, Sec. Separation of Concerns], [25, Sec. Focus on Domain].

4.2 Identifizierte Herausforderungen bei der Navigation

4.2.1 Verteilte Artefakte und implizite Abhängigkeiten

Ein CAP-Service erstreckt sich über mehrere konzeptionelle Ebenen, wobei Modell, Schnittstelle, Implementierung und Sicherheitsregeln physisch getrennt sind. Die Beziehungen zwischen diesen Artefakten sind nicht visuell auf einen Blick erkennbar. Projektionen referenzieren Entities, Event Handler referenzieren Service-Operationen, und Sicherheitsannotationen beeinflussen das Verhalten ohne zentrale Sichtbarkeit. Kein einzelnes Artefakt enthält die vollständige Service-Definition.

Diese Problematik entspricht der von Tarr et al. beschriebenen „Tyrannei der dominanten Dekomposition“ [26, p. 809]: Software wird typischerweise entlang einer primären Dimension strukturiert. Im Fall von CAP-Projekten nach Ordnern und Dateien entsprechend technischer Verantwortlichkeiten. Andere wichtige Dimensionen, wie die servicezentrierte fachliche Sicht, werden dadurch über das Projekt verteilt und sind nicht mehr unmittelbar sichtbar [3].

4.2.2 Erhöhter Navigationsaufwand und fehlende Service-Übersicht

Mit zunehmender Weiterentwicklung eines Softwaresystems steigt dessen strukturelle Komplexität. Lehman beschreibt dieses Phänomen im Gesetz der zunehmenden Komplexität, wonach die Komplexität eines Systems mit fortlaufenden Änderungen wächst, sofern keine Maßnahmen zur Reduktion dieser Komplexität ergriffen werden [27, p. 1068]. Zur Analyse eines Services sind mehrere Dateiwechsel erforderlich, etwa zwischen Datenbankmodell, Implementierung und Sicherheitsregeln. Da keine aggregierte Sicht auf alle servicebezogenen Artefakte existiert, muss die Struktur eines Services rekonstruiert werden. Diese Kontextwechsel erhöhen die kognitive Belastung, da Menschen nur über eine begrenzte kognitive Verarbeitungskapazität verfügen und mehrere Informationen gleichzeitig berücksichtigen müssen [28, p. 261].

4.2.3 Quantifizierung anhand konkreter Szenarien

Die beschriebenen Navigationsprobleme lassen sich anhand typischer Entwicklungsaufgaben im Bookshop-Beispielprojekt [4] konkretisieren. Abbildung 4 zeigt

die Navigationskomplexität für vier repräsentative Szenarien, ermittelt durch manuelle Analyse des Projekts. Die detaillierten Navigationspfade mit konkreten Dateien und Zeilennummern sind im Anhang dokumentiert.

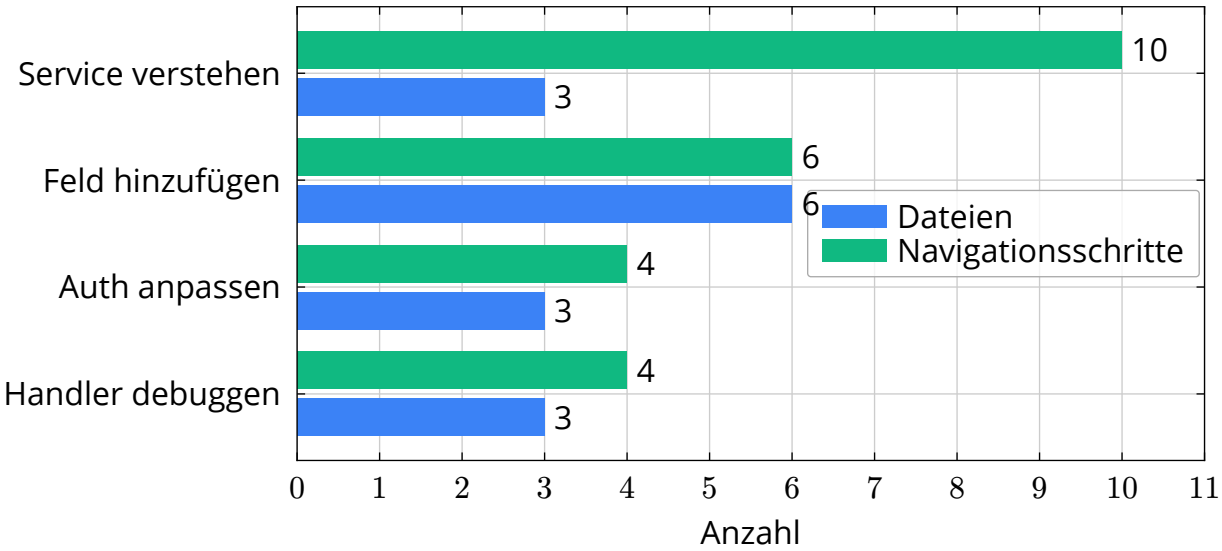

Abbildung 4 — Navigationskomplexität typischer Entwicklungsaufgaben im Bookshop-Projekt

Das Szenario „Handler debuggen“ zeigt exemplarisch einen typischen Debugging-Workflow: Ein Entwickler untersucht das Verhalten der `submitOrder`-Action im CatalogService. Er öffnet zunächst `srv/cat-service.js` (Zeile 14), analysiert die Handler-Logik, navigiert zur Action-Definition in `srv/cat-service.cds` (Zeile 19) und prüft das referenzierte Entity `Books` in `db/schema.cds`. Dieser Workflow erfordert 3 Dateien und 4 Navigationsschritte.

Das komplexeste Szenario „Service verstehen“ erfordert das Nachvollziehen aller Aspekte des CatalogService: Service-Definition mit allen Projektionen und Actions (`srv/cat-service.cds`), das vollständige Datenmodell mit allen referenzierten Entities (`db/schema.cds`) sowie die Handler-Implementierung (`srv/cat-service.js`). Obwohl nur 3 Dateien beteiligt sind, erfordert das Verständnis 10 Navigationsschritte zwischen verschiedenen Artefakten.

Abbildung 2 visualisiert diese Fragmentierung am Beispiel des CatalogService.

└─ **CatalogService** (fachliche Einheit)

→`db/schema.cds` — Domänenmodell (Books, Authors, Genres, Price)

→`srv/cat-service.cds` — Service-Definition, Projektionen, Action

└─ `srv/cat-service.js` — Event Handler (after, submitOrder)

3 Dateien in **2 Verzeichnissen** — keine aggregierte Sicht verfügbar

Abbildung 2 — Verteilung der CatalogService-Artefakte im Bookshop-Projekt

4.2.4 Bestehende Ansätze zur Navigation in Entwicklungsumgebungen

Visual Studio Code bietet grundlegende Navigationsfunktionen wie die Outline-Ansicht [29, Sec. Outline View] und Go-to-Definition [30, Sec. Go to Definition]. Die SAP CDS Language Support Extension [31] erweitert dies um CDS-spezifische Features wie Syntax-Highlighting und Autocomplete. Forschung zu Informationsbedürfnissen in Softwareteams zeigt, dass Entwickler häufig Schwierigkeiten haben, relevante Informationen über Code, Design und Programmausführung aus verschiedenen Artefakten zusammenzuführen [32, p. 347]. Diese Werkzeuge operieren überwiegend datei- und symbolzentriert, sodass eine servicezentrierte Aggregation über mehrere Dateien und Ebenen hinweg nicht unterstützt wird.

4.3 Ableitung der Designziele

Aus den identifizierten Navigationsproblemen werden nun zentrale Designziele abgeleitet. Diese beschreiben lösungsneutral, welche Eigenschaften eine geeignete Lösung aufweisen muss. Die Designziele bilden die Grundlage für die anschließende Definition funktionaler und nicht-funktionaler Anforderungen.

Aggregierte Service-Sicht

Alle Artefakte eines Services müssen gebündelt dargestellt werden. Die Lösung muss eine einheitliche Einstiegsperspektive auf einen Service bieten, in der Modell, Schnittstelle, Implementierung und Sicherheitsaspekte gemeinsam sichtbar sind. Die Service-Identität muss ohne manuelle Rekonstruktion erfassbar sein.

Reduktion von Navigations- und Kontextwechseln

Die Anzahl notwendiger Dateiwechsel muss minimiert werden. Deklarative und imperative Artefakte müssen kontextnah zugänglich sein. Relevante Artefakte müssen schnell auffindbar sein. Kognitive Kontextwechsel zwischen den Ebenen müssen reduziert werden.

Explizite Darstellung von Abhängigkeiten

Beziehungen zwischen allen Arten von Entitäten müssen sichtbar gemacht werden. Verknüpfungen zwischen Service-Definition und Implementierung müssen nachvollziehbar sein. Sicherheitsrelevante Annotationen müssen eindeutig zuordenbar sein. Referenzen müssen transparent werden.

Servicezentrierte Navigation

Die Navigation muss sich an den fachlichen Services orientieren. Die technische Ordnerstruktur darf nicht primärer Navigationsanker sein. Der Service muss als logische Einheit im Mittelpunkt stehen.

4.4 Funktionale Anforderungen

Aus den zuvor formulierten Designzielen lassen sich konkrete funktionale Anforderungen an die zu entwickelnde VS-Code Extension ableiten. Während die Designziele die gewünschten Eigenschaften der Lösung auf konzeptioneller Ebene beschreiben, spezifizieren die folgenden Anforderungen die Funktionen, die das Artefakt zur Unterstützung der Navigation in CAP-Projekten bereitstellen muss.

FA1 Identifikation von CAP-Services

Die Extension muss CAP-Services innerhalb eines Projekts automatisch identifizieren und alle Services anzeigen.

FA2 Aggregierte Serviceübersicht

Die Extension muss eine aggregierte Darstellung aller zu einem Service gehörenden Artefakte bereitstellen.

FA3 Navigation zu Artefakten

Die Extension muss eine direkte Navigation von der Serviceübersicht zu den entsprechenden Artefakten im Quellcode ermöglichen.

FA4 Darstellung von Artefaktbeziehungen
Die Extension muss Beziehungen und Hierarchien zwischen servicebezogenen Artefakten sichtbar machen.

4.4.1 Nicht-funktionale Anforderungen

Neben den funktionalen Anforderungen ergeben sich aus dem Nutzungskontext innerhalb der Entwicklungsumgebung zusätzliche nicht-funktionale Anforderungen. Diese beschreiben qualitative Eigenschaften und Rahmenbedingungen, die bei der Umsetzung der Extension berücksichtigt werden müssen.

NFA1 Integration in Visual Studio Code
Die Lösung muss als Erweiterung für die Entwicklungsumgebung Visual Studio Code implementiert werden und sich in deren Erweiterungsmechanismus [15] integrieren.

NFA2 Kompatibilität mit CAP-Projektstrukturen
Die Extension muss mit der typischen Struktur von SAP CAP-Projekten kompatibel sein und relevante Artefakte in den vorgesehenen Verzeichnissen erkennen können.

NFA3 Performante Analyse von Projekten
Die Analyse eines CAP-Repositories soll ohne merkliche Verzögerung erfolgen, sodass die Nutzung der Extension den Entwicklungsworkflow nicht beeinträchtigt.

NFA4 Aktualität der Artefaktübersicht
Änderungen an relevanten Projektartefakten müssen automatisch erkannt werden, sodass die dargestellte Serviceübersicht den aktuellen Projektzustand widerspiegelt.

4.5 Abgrenzung der Arbeit

Die vorliegende Arbeit fokussiert auf die Unterstützung der Navigation in CAP-Projekten durch eine VS-Code Extension. Daraus ergeben sich bewusste Einschränkungen des Untersuchungs- und Lösungsumfangs.

Ziel der Extension ist die Verbesserung der Navigation zwischen servicebezogenen Artefakten. Funktionen zur automatischen Codegenerierung oder Refaktorierung sind explizit nicht Bestandteil der Lösung.

Die Evaluation erfolgt anhand ausgewählter CAP-Projekte. Eine umfassende Nutzerstudie mit CAP-Entwicklern ist nicht Bestandteil der Arbeit.

5 Integrationsstrategien

Dieses Kapitel stellt verschiedene Integrationsstrategien für die Implementierung der Service-Struktur-Analyse vor. Es entspricht der DSR-Phase „Artefaktentwurf“. Das Ziel ist die systematische Auswahl einer geeigneten Strategie basierend auf den Anforderungen aus Kapitel 4.

5.1 Repository-basierte Integration

Die repository-basierte Integration rekonstruiert die Service-Struktur direkt aus den Dateien des Projektrepositories. Die Analyse erfolgt durch Parsing der `.cds`-Dateien, wobei Entities, Services, Projektionen und Operationen syntaktisch extrahiert werden. Die Implementierungslogik wird durch Analyse von `.js`- und `.ts`-Dateien ermittelt. Beziehungen zwischen Artefakten müssen aus Referenzen im Code rekonstruiert werden. Quellcode 5 zeigt einen Ausschnitt einer typischen CDS-Service-Definition.

```
using { sap.capire.bookshop as my } from ‚../db/schema‘

service CatalogService {
  @readonly entity Books as projection on my.Books {
    *, author.name as author, genre.name as genre
  } excluding { createdBy, modifiedBy };

  @requires: ‚authenticated-user‘
  action submitOrder ( book: Books:ID, quantity: Integer );
}
```

Quellcode 5 — CDS-Service-Definition mit Projektionen und Action

Die technische Umsetzung erfordert einen eigenen Parser für die CDS-Sprache. CDS ist eine domänenspezifische Sprache mit eigener Grammatik, die Konstrukte wie `entity`, `service`, `projection`, `aspect` und `type` unterstützt. Wie Quellcode 5 zeigt, enthält bereits eine einfache Service-Definition mehrere Parsing-Herausforderun-

gen: den `using`-Import mit Alias, verschachtelte Projektionen mit Feld-Aliasing (`author.name as author`), Annotationen (`@readonly`, `@requires`) sowie Actions mit Parametern. Ein Parser müsste all diese Konstrukte erkennen, verschachtelte Strukturen auflösen und Referenzen zwischen Dateien verfolgen. Zusätzlich müssten `.js`- und `.ts`-Dateien analysiert werden, um Event Handler den entsprechenden Services zuzuordnen.

Dieser Ansatz bringt einen hohen Implementierungsaufwand für Parsing und Referenzauflösung mit sich. Zudem besteht das Risiko semantischer Fehlinterpretationen der CDS-Sprache, da ein eigener Parser die Sprachsemantik nicht vollständig abbilden kann. Darüber hinaus dupliziert dieser Ansatz Analysefunktionen, die bereits durch das CAP-Tooling bereitgestellt werden. Der Vorteil dieses Ansatzes liegt in seiner hohen Flexibilität. Ein eigener Parser ermöglicht die vollständige Kontrolle über die extrahierten Informationen und ist unabhängig von externen Tools.

5.2 CSN-basierte Integration

Die CSN-basierte Integration nutzt das von CAP generierte Core Schema Notation (CSN) Modell als Informationsquelle. CSN ist eine kompilierte JSON-Repräsentation aller CDS-Artefakte. Der CAP-Compiler erzeugt CSN aus den `.cds`-Quelldateien und löst dabei alle Referenzen, Imports und Vererbungen auf.

Im CSN-Modell sind Entities, Services, Projektionen und Operationen strukturiert enthalten. Jedes Artefakt ist unter seinem vollqualifizierten Namen als JSON-Objekt abgelegt. Das Feld `kind` gibt den Artefakttyp an (z.B. `entity`, `service`, `function`). Beziehungen zwischen Artefakten sind bereits explizit aufgelöst, sodass keine eigene Referenzauflösung erforderlich ist. Quellcode 6 zeigt einen Ausschnitt eines CSN-Modells.

```
{
  „definitions“: {
    „TravelService“: {
      „kind“: „service“,
      „$location“: { „file“: „srv/travel-service.cds“, „line“: 5 }
    },
    „TravelService.Travels“: {
      „kind“: „entity“,
      „projection“: {
        „from“: { „ref“: [„sap.capire.travels.Travels“] }
      },
      „elements“: {
        „ID“: { „key“: true, „type“: „cds.Integer“ },
        „Description“: { „type“: „cds.String“, „length“: 1024 },
        „BeginDate“: { „type“: „cds.Date“ }
      }
    }
  }
}
```

Quellcode 6 — Ausschnitt eines CSN-Modells mit Service und Entity

Die semantisch korrekte Interpretation ist durch die Nutzung des CAP-Compilers garantiert. Das JSON-Format ist direkt parsebar und erfordert keinen eigenen CDS-Parser. CSN bietet optional Positionsinformationen über das Feld `$location`, die durch einen speziellen Kompilierungsmodus aktiviert werden können. Diese Positionsinformationen sind jedoch unvollständig: Nicht alle Artefakte und Elemente erhalten Location-Daten. Für eine zuverlässige Navigation zu Quellcode-Positionen (FA3) ist diese Abdeckung nicht ausreichend. Zusätzlich ist CSN integraler Bestandteil des CAP-Compilers. Erweiterungen am CSN-Format würden den Build-Prozess verlangsamen und sind daher nicht praktikabel.

5.3 Language-Server-basierte Integration

Die Language-Server-basierte Integration nutzt den CAP Language Server als zentrale Analysequelle. Der Language Server implementiert das Language Server

Protocol (LSP) und stellt semantische Analysen der CDS-Artefakte bereit. Er läuft als separater Prozess und kommuniziert mit VS Code über JSON-RPC.

Für die Service-Struktur-Analyse ist der LSP-Request `workspace/symbol` relevant. Dieser Request liefert alle Symbole eines Workspace in einem strukturierten JSON. Jedes Symbol enthält den Namen, den Container (übergeordnetes Element), die Datei-URI, die exakte Position (range) im Quellcode sowie den Symboltyp (kind). Quellcode 7 zeigt einen Ausschnitt der Antwort des CAP Language Servers.

```
[
  {
    „name“: „Bookings“,
    „containerName“: „TravelService“,
    „location“: {
      „uri“: „file:///srv/travel-service.cds“,
      „range“: {
        „start“: { „line“: 4, „character“: 0 },
        „end“: { „line“: 21, „character“: 1 }
      }
    },
    „kind“: 5
  },
  {
    „name“: „BookingDate“,
    „containerName“: „TravelService.Bookings“,
    „location“: { … },
    „kind“: 8
  }
]
```

Quellcode 7 — Beispiel einer Workspace-Symbol-Antwort des CAP Language Servers

Das Feld `containerName` kodiert die Hierarchie der Artefakte. Ein Wert wie `TravelService.Bookings` bedeutet, dass das Symbol `BookingDate` innerhalb von `Bookings` liegt, welches wiederum zum `TravelService` gehört. Das Feld `kind` gibt den Symboltyp (kind) an: 5 steht für eine Entity, 8 für ein Feld, 12 für eine Funktion und

19 für einen Service. Diese Information ermöglicht die Rekonstruktion der Service-Hierarchie.

Der Language Server bietet eine native Integration in VS Code über das Language Server Protocol. Bei Bedarf ist der Language Server erweiterbar, da diese Arbeit in der Abteilung entsteht, die den Language Server entwickelt. Die Strategie setzt jedoch voraus, dass der Language Server zur Laufzeit verfügbar ist.

5.4 Vergleich und Auswahl

Im Rahmen dieser Bachelorarbeit wird nur eine Strategie implementiert. Zur Auswahl wird eine Nutzwertanalyse durchgeführt. Die Strategie mit dem höchsten Nutzwert wird in Kapitel 6 umgesetzt.

5.4.1 Nutzwertanalyse als Bewertungsmethode

Die Nutzwertanalyse ist ein Verfahren zur Bewertung von Entscheidungsalternativen bei mehreren Zielgrößen, bei dem qualitative und quantitative Kriterien systematisch in eine Entscheidung einbezogen werden [33, pp. 193–194]. Das Verfahren eignet sich besonders für Entscheidungsprobleme, bei denen nicht alle Kriterien quantitativ messbar sind, wie es bei der Bewertung von Softwarearchitekturen der Fall ist.

Die Methode funktioniert wie folgt [33, pp. 193–194]: Zunächst werden relevante Bewertungskriterien definiert und mit Gewichten versehen, die ihre relative Bedeutung ausdrücken. Die Summe aller Gewichte beträgt 1. Jede Alternative wird anschließend bezüglich jedes Kriteriums mit einem Punktwert auf einer definierten Skala bewertet. Der Gesamtnutzwert einer Alternative ergibt sich aus der Summe der gewichteten Einzelbewertungen. Die Alternative mit dem höchsten Nutzwert wird als vorteilhafteste Option identifiziert.

Der Gesamtnutzwert N_{Ni} einer Alternative i berechnet sich formal wie folgt [33, p. 196]:

$$N_{Ni} = \sum_{k=1}^{K} n_{ik} \cdot w_k \tag{1}$$

Dabei bezeichnet n_{ik} den Teilnutzenwert der Alternative i bezüglich des Kriteriums k, w_k das Gewicht des Kriteriums k und K die Anzahl der Bewertungskriterien.

Im Folgenden wird die Nutzwertanalyse auf die Bewertung der drei Integrationsstrategien angewendet. Die Gewichtung der Kriterien und die Bewertung der Alternativen erfolgen auf Basis der in Kapitel 4 identifizierten Anforderungen.

5.4.2 Bewertungskriterien und Gewichtung

Für die Nutzwertanalyse werden Bewertungskriterien herangezogen. Diese leiten sich von den Anforderungen ab oder werden als sinnvoll erachtet. Tabelle 2 zeigt diese Bewertungskriterien mit ihrer Gewichtung und Herleitung aus den Anforderungen. In den Abschnitten darunter werden die Kriterien näher erläutert.

Kriterium	Gewicht	Herleitung
K1: Semantische Genauigkeit	25%	FA1, FA2
K2: Positionsinformationen	25%	FA3
K3: VS Code Integration	20%	NFA1
K4: Implementierungsaufwand	15%	–
K5: Wartbarkeit	10%	NFA4
K6: Unabhängigkeit	5%	–

Tabelle 2 — Bewertungskriterien und Gewichtung

K1: Semantische Genauigkeit (25%) beschreibt die Fähigkeit, CDS-Artefakte korrekt zu identifizieren und zu interpretieren. Das Kriterium leitet sich aus FA1 und FA2 ab, da Services korrekt erkannt werden müssen. Es erhält die höchste Gewichtung, da ohne korrekte Erkennung die Extension nicht nutzbar ist.

K2: Positionsinformationen (25%) bezeichnet die Verfügbarkeit von Quellcode-Positionen (Zeile/Spalte) für Artefakte. Das Kriterium leitet sich aus FA3 ab, da Navigation zu Definitionen möglich sein muss. Es erhält ebenfalls die höchste Gewichtung, da Navigation das zentrale Feature der Extension ist.

K3: VS Code Integration (20%) bewertet die Kompatibilität mit VS Code APIs und der Erweiterungsarchitektur. Das Kriterium leitet sich aus NFA1 ab und fordert eine nahtlose Einbindung in die IDE. Die hohe Gewichtung ergibt sich daraus, dass schlechte Integration die Akzeptanz mindert.

K4: Implementierungsaufwand (15%) erfasst Komplexität und Zeitbedarf für die Umsetzung. Es gibt keine direkte Anforderung, jedoch besteht eine praktische Ressourcenbeschränkung im Rahmen dieser Bachelorarbeit. Die mittlere Gewichtung spiegelt den realistischen Projektrahmen wider.

K5: Wartbarkeit (10%) bewertet die langfristige Anpassbarkeit bei Änderungen am CAP-Framework. Das Kriterium leitet sich aus NFA4 ab, da die Extension auch zukünftig funktionieren soll. Die geringere Gewichtung ergibt sich daraus, dass zunächst die Funktionalität wichtiger ist.

K6: Unabhängigkeit (5%) bezeichnet die Freiheit von externen Laufzeitabhängigkeiten. Es gibt keine direkte Anforderung. Abhängigkeiten sind akzeptabel, wenn die Funktionalität dadurch besser wird. Die niedrigste Gewichtung ist ein bewusster Trade-off.

Tabelle 3 zeigt die Bewertung der drei Strategien anhand der definierten Kriterien.

Kriterium	Gewicht	Repository	CSN	Language Server
K1: Semantische Genauigkeit	25%	2	5	5
K2: Positionsinformationen	25%	3	1	5
K3: VS Code Integration	20%	2	3	5
K4: Implementierungsaufwand	15%	2	4	3
K5: Wartbarkeit	10%	2	3	4
K6: Unabhängigkeit	5%	5	2	2
Gewichtete Summe	100%	**2.40**	**3.10**	**4.45**

Tabelle 3 — Nutzwertanalyse der Integrationsstrategien

Punkteskala: 1 = unzureichend | 2 = ausreichend | 3 = befriedigend | 4 = gut | 5 = sehr gut

Die detaillierte Begründung der Punktvergabe für jedes Kriterium und jede Strategie ist in Tabelle 4 im Anhang dokumentiert. Die wesentlichen Unterschiede

lassen sich wie folgt zusammenfassen: Bei K1 (Semantische Genauigkeit) erzielen CSN und Language Server Höchstwerte, da beide den CAP-Compiler nutzen. Der entscheidende Differenzierungsfaktor ist K2 (Positionsinformationen): CSN liefert diese nur unvollständig, während der Language Server über die Location-API präzise Quellcode-Positionen bereitstellt. Bei K3 (VS Code Integration) profitiert der Language Server von der nativen LSP-Unterstützung, während Repository und CSN zusätzliche Adapter erfordern. Der Implementierungsaufwand (K4) ist bei CSN am geringsten, da das JSON-Format direkt parsebar ist.

5.5 Auswahl der Integrationsstrategie

Die Nutzwertanalyse ergibt den höchsten Wert für die Language-Server-basierte Integration mit 4.45 Punkten (Abbildung 8). Der Language Server bietet die beste VS Code Integration, da er das Language Server Protocol nativ implementiert. Er liefert vollständige Positionsinformationen, während CSN diese nur unvollständig bereitstellt. Ein eigener Parser wäre aufwendig und fehleranfällig. Zusätzlich kann der Language Server bei Bedarf erweitert werden, da diese Arbeit in der Abteilung entsteht, die ihn entwickelt.

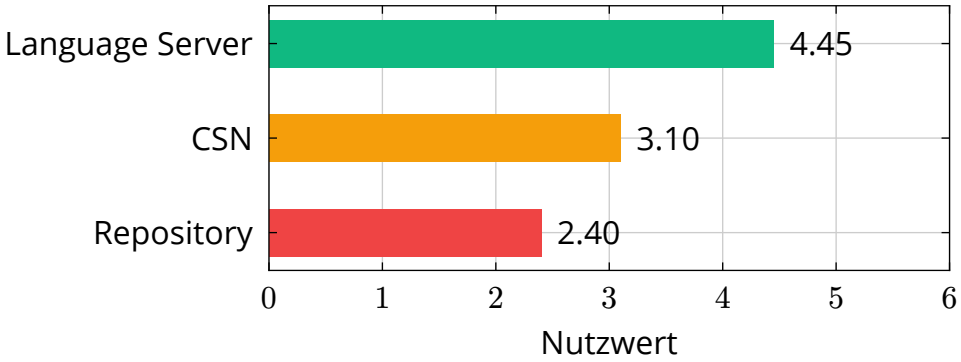

Abbildung 8 — Ergebnis der Nutzwertanalyse

Die Wahl des Language Servers bringt eine Abhängigkeit zur Laufzeit mit sich. Dieser Trade-off wird zugunsten der höheren Funktionalität akzeptiert. Das folgende Kapitel beschreibt die Implementierung der Extension auf Basis der gewählten Language-Server-Integration.

6 Implementierung des Prototyps

Im folgenden Kapitel wird ein Prototyp basierend auf der Language-Server-basierten Strategie implementiert. Der Fokus liegt auf einer aggregierten Service-Übersicht zur Navigation sowie einer Suchfunktion mit Filteroptionen.

6.1 Architekturkonzept und Designentscheidungen

6.1.1 Architektur

Abbildung 9 zeigt die Komponentenstruktur der Extension. Die Extension läuft innerhalb des VS Code Extension Host und kommuniziert mit zwei externen Systemen. Der CDS Language Server dient als Datenquelle und die VS Code TreeView API ermöglicht die UI-Darstellung. Diese klare Trennung zwischen internen Extension-Komponenten und externen Abhängigkeiten ermöglicht modulare Architektur.

Das Modul `extension.ts` dient als zentraler Orchestrator. Es instanziiert alle Komponenten und verdrahtet deren Kommunikation. Dieser Ansatz folgt dem Dependency Injection Pattern, bei dem Komponenten sich nicht direkt kennen, sondern über den Orchestrator kommunizieren. Dadurch bleiben Abhängigkeiten explizit und testbar.

Die Extension besteht aus vier gleichberechtigten Hauptkomponenten. Der `cdsClient` beschafft Daten vom CDS Language Server. Der `TreeProvider` rendert die Baumstruktur in der VS Code UI. Der `Controller` verwaltet den Anwendungszustand und koordiniert die Business Logic. Der `SearchProvider` stellt die Suchoberfläche als Webview bereit. Jede Komponente hat eine klar definierte Verantwortlichkeit.

Unterhalb des Controllers liegt der Domain Layer mit den Modulen `treeBuilder`, `search` und `filter`. Diese enthalten die gesamte Business Logic für Datentransformation und Filterung. Der Domain Layer hat keine Abhängigkeit zur VS Code API

und ist dadurch isoliert testbar. Dies entspricht dem Prinzip der Separation of Concerns.

Der Datenfluss verläuft vom CDS Language Server über den `cdsClient` zum `Controller`, der die Domain-Module für Transformation nutzt. Das Ergebnis geht an den `TreeProvider` zur Darstellung. User Input vom `SearchProvider` fließt zurück zum `Controller`, der über den Domain Layer filtert und aktualisierte Daten an den `TreeProvider` sendet.

Die Architektur folgt dem Single Responsibility Principle, bei dem Datenakquise, State Management und Darstellung getrennt sind. Dies verbessert Wartbarkeit, ermöglicht isoliertes Testen einzelner Komponenten und erleichtert zukünftige Erweiterungen.

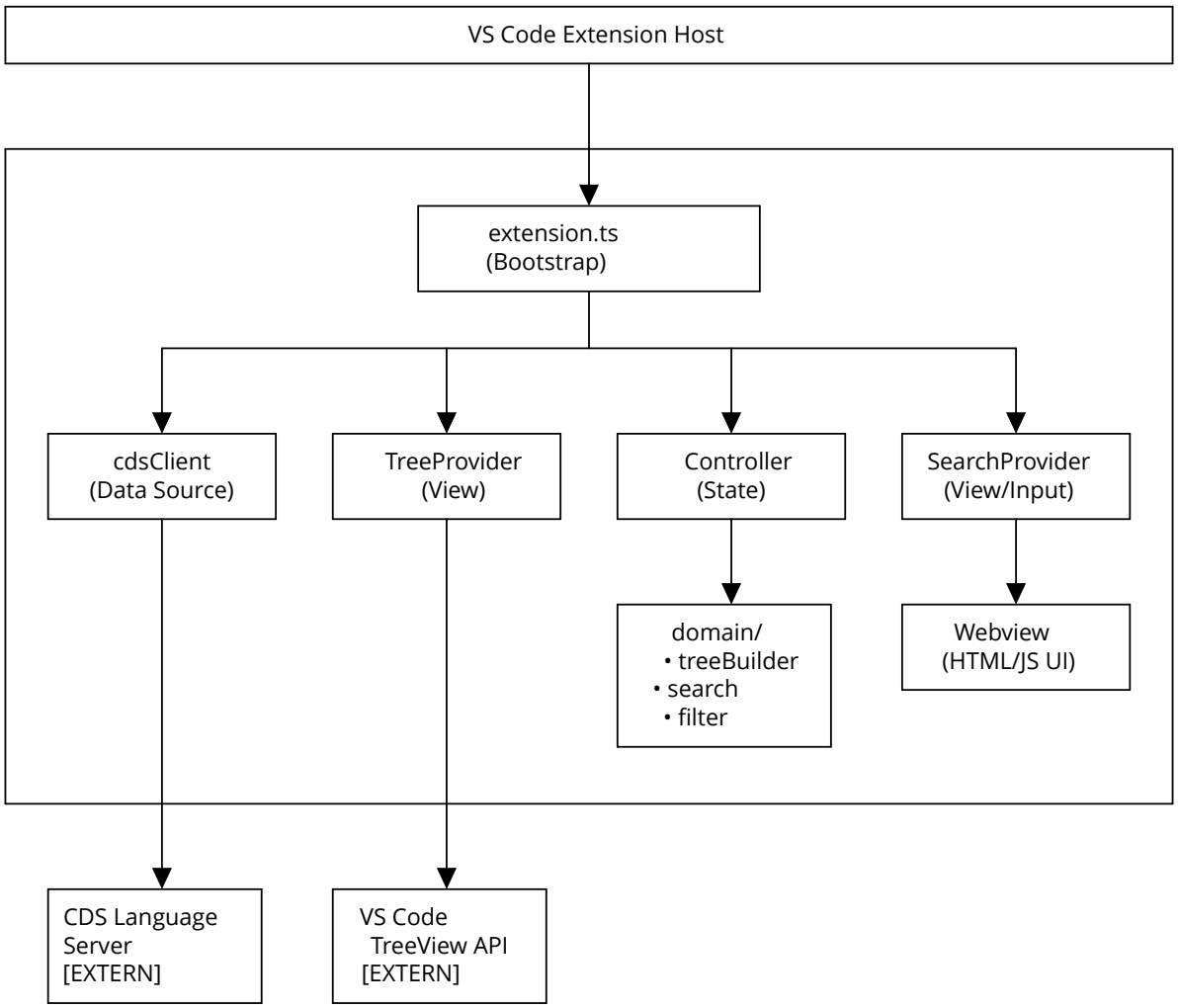

Abbildung 9 — Komponentenstruktur und Datenfluss der VS Code Extension

6.1.2 Technologie-Stack

Die Extension wurde vollständig in TypeScript implementiert. Die VS Code Extension API ist in TypeScript geschrieben und bietet vollständige Typisierung. Type Safety zur Compile-Zeit reduziert Fehler, IDE-Unterstützung (Autocomplete, Refactoring) verbessert Entwicklungsgeschwindigkeit. Alternative JavaScript wäre möglich, bietet aber keine Typsicherheit und erhöht das Fehlerrisiko.

Das Suchpanel wurde als Webview mit HTML/CSS/JavaScript implementiert. Native VS Code UI-Komponenten (Input Box, QuickPick) sind zu limitiert für die komplexen UI-Anforderungen (Suchfeld + Filterchips + Optionen-Checkboxen). Webview

ermöglicht volle Gestaltungsfreiheit, persistente UI-Darstellung und Nutzung moderner Web-Technologien. Auf Frontend-Frameworks (React, Vue, Svelte) wurde bewusst verzichtet, da das UI sehr simpel ist und natives HTML/CSS/JavaScript bessere Performance bietet ohne zusätzlichen Bundle-Overhead.

Der Build-Prozess wird über npm gesteuert, das sowohl Package Management als auch Build-Orchestrierung übernimmt. Alternative Package Manager wie Yarn oder pnpm wären möglich, npm wurde jedoch gewählt, da es als Standard-Tool in Node.js integriert ist und keine zusätzliche Installation erfordert. npm scripts definieren den gesamten Build-Workflow (Kompilierung, Watch-Modus, Packaging). Die finale Extension wird als VSIX-Datei (VS Code Extension Package Format) ausgeliefert. Diese Toolchain entspricht dem Standard für VS Code Extensions und erfordert keine komplexen Build-Tools.

6.2 Integration mit CDS Language Server

6.2.1 Kommunikation über Language Server Protocol

- Extension kommuniziert mit CDS Language Server über Language Server Protocol (LSP)
- LSP ist standardisiertes Protokoll für Kommunikation zwischen Editor und Language Server
- Zugriff auf CDS Language Server über SAP CDS Extension (`sapse.vscode-cds`)
- SAP CDS Extension exponiert Language Client über API (`getLanguageClient()`)
- Extension prüft Verfügbarkeit der CDS Extension vor jeder Anfrage
- `workspace/symbol` Request als zentrale LSP-Operation
  - Liefert alle CDS-Symbole des Workspace (Entities, Services, Actions, Types, etc.)
  - Query-Parameter ermöglicht Filterung (z.B. `/gax` für alle Symbole)
  - Response enthält Symbol-Name, Kind, Location (Datei + Position), Container-Name
- Fehlerbehandlung für fehlende oder inaktive CDS Extension

- Warum LSP statt direktem Parsing: Wiederverwendung existierender Analyse-Infrastruktur, semantische Korrektheit garantiert

6.2.2 Normalisierung der Symboldaten

- LSP liefert `WorkspaceSymbol[]` mit heterogener Struktur (LSP-spezifisches Format)
- Normalisierung zu einheitlichem `Symbol` Interface der Extension
- `normalizeSymbol()` Funktion konvertiert LSP-Format zu Domain-Modell
- Extrahierte Felder:
  - `name`: Symbol-Name (String)
  - `kind`: Symbol-Typ als numerischer Code (Entity = 5, Service = 19, Action = 12, etc.)
  - `containerName`: Hierarchischer Kontext (z.B. „ServiceName.EntityName“)
  - `fileUri`: Dateipfad des Symbols
  - `range`: Position im Quellcode (Zeile, Spalte, Start, Ende)
  - `flags`: Optional, zusätzliche Metadaten (Bitmap für autoexposed, etc.)
- Validierung und Default-Werte für fehlende Felder
- Symbol-Typen definiert in `SymbolKind` Konstanten (ENTITY, SERVICE, ACTION_FUNC, ELEMENT, etc.)
- Normalisierte Symbole dienen als Basis für Tree-Building
- Trennung LSP-Format von Domain-Modell ermöglicht unabhängige Evolution

6.3 Datenmodell und Tree-Building

6.3.1 Symbol- und TreeNode-Typen

6.3.2 Tree-Building-Strategie

6.4 State Management und Visualisierung

6.4.1 Controller und ViewState

6.4.2 TreeDataProvider und VS Code Integration

6.5 Such- und Filtersystem

6.6 Navigation und Interaktion

7 Evaluation

7.1 Evaluationskriterien

7.2 Vergleich der Integrationsansätze

7.3 Analyse der Ergebnisse

8 Diskussion und Fazit

8.1 Interpretation der Ergebnisse

8.2 Limitationen

8.3 Zusammenfassung der Ergebnisse

8.4 Beantwortung der Forschungsfragen

8.5 Ausblick auf zukünftige Arbeiten

Literatur

[1] SAP, „Concepts Overview | capire“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: https://cap.cloud.sap/docs/get-started/concepts

[2] SAP, „Core Data Services (CDS) | capire“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: https://cap.cloud.sap/docs/cds/

[3] SAP, „Domain Modeling | capire“. Zugegriffen: 10. März 2026. [Online]. Verfügbar unter: https://cap.cloud.sap/docs/guides/domain

[4] capire, „GitHub – capire/bookshop: Our primer sample to get started in a nutshell“. Zugegriffen: 25. Februar 2026. [Online]. Verfügbar unter: https://github.com/capire/bookshop

[5] SAP, „About CAP | capire“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: https://cap.cloud.sap/docs/

[6] O. Nierstrasz und F. Achermann, „Separation of Concerns through Unification of Concepts“, in *ECOOP 2000 Workshop on Aspects & Dimensions of Concerns*, Bern, Switzerland, 2000.

[7] M. P. Papazoglou, „Service-Oriented Computing: Concepts, Characteristics and Directions“, in *Proceedings of the Fourth International Conference on Web Information Systems Engineering (WISE)*, IEEE, 2003, S. 3–12. doi: 10.1109/WISE.2003.1254461.

[8] OASIS, „OData Version 4.01“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: https://www.odata.org/documentation/

[9] OpenAPI Initiative, „OpenAPI Specification, Version 3.2.0“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: https://spec.openapis.org/oas/latest.html

[10] SAP, „Definition Language (CDL) | capire“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: https://cap.cloud.sap/docs/cds/cdl

[11] SAP, „Core Schema Notation (CSN) | capire“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: https://cap.cloud.sap/docs/cds/csn

[12] Sourcegraph, „Langserver.org – A community-driven source of knowledge for Language Server Protocol implementations“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: https://langserver.org/

[13] JSON-RPC Working Group, „JSON-RPC 2.0 Specification“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: https://www.jsonrpc.org/specification

[14] Microsoft, „Language Server Protocol Specification“. Zugegriffen: 30. März 2026. [Online]. Verfügbar unter: https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/

[15] Microsoft, „Extension API | Visual Studio Code“. Zugegriffen: 10. März 2026. [Online]. Verfügbar unter: https://code.visualstudio.com/api

[16] Microsoft, „Extension Capabilities Overview | Visual Studio Code Extension API“. Zugegriffen: 13. April 2026. [Online]. Verfügbar unter: https://code.visualstudio.com/api/extension-capabilities/overview

[17] Microsoft, „Tree View API | Visual Studio Code Extension API“. Zugegriffen: 13. April 2026. [Online]. Verfügbar unter: https://code.visualstudio.com/api/extension-guides/tree-view

[18] Microsoft, „Language Server Extension Guide | Visual Studio Code Extension API“. Zugegriffen: 13. April 2026. [Online]. Verfügbar unter: https://code.visualstudio.com/api/language-extensions/language-server-extension-guide

[19] A. R. Hevner, S. T. March, J. Park, und S. Ram, „Design Science in Information Systems Research“, *MIS Quarterly*, Bd. 28, Nr. 1, S. 75–105, 2004, [Online]. Verfügbar unter: https://www.jstor.org/stable/25148625

[20] R. L. Baskerville und A. T. Wood-Harper, „A Critical Perspective on Action Research as a Method for Information Systems Research“, *Journal of Information Technology*, Bd. 11, Nr. 3, S. 235–246, 1996, doi: 10.1080/026839696345289.

[21] P. Runeson und M. Höst, „Guidelines for Conducting and Reporting Case Study Research in Software Engineering“, *Empirical Software Engineering*, Bd. 14, Nr. 2, S. 131–164, 2009, doi: 10.1007/s10664-008-9102-8.

[22] A. Dresch, D. P. Lacerda, und J. A. V. Antunes Jr., *Design Science Research: A Method for Science and Technology Advancement*. Springer, 2014. doi: 10.1007/978-3-319-07374-3.

[23] K. Peffers, T. Tuunanen, M. A. Rothenberger, und S. Chatterjee, „A Design Science Research Methodology for Information Systems Research“, *Journal of Management Information Systems*, Bd. 24, Nr. 3, S. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.

[24] SAP, „The Bookshop Sample – Separation of Concerns | capire“. Zugegriffen: 10. März 2026. [Online]. Verfügbar unter: https://cap.cloud.sap/docs/get-started/bookshop#separation-of-concerns

[25] SAP, „The Bookshop Sample – Focus on Domain | capire“. Zugegriffen: 10. März 2026. [Online]. Verfügbar unter: https://cap.cloud.sap/docs/get-started/bookshop#focus-on-domain

[26] P. Tarr, W. Harrison, H. Ossher, A. Finkelstein, B. Nuseibeh, und D. Perry, „Workshop on Multi-Dimensional Separation of Concerns in Software Engineering“, in *Proceedings of the 22nd International Conference on Software Engineering (ICSE)*, Limerick, Ireland: ACM, 2000, S. 809–810.

[27] M. M. Lehman, „Programs, Life Cycles, and Laws of Software Evolution“, *Proceedings of the IEEE*, Bd. 68, Nr. 9, S. 1060–1076, 1980, doi: 10.1109/PROC.1980.11805.

[28] J. Sweller, „Cognitive Load During Problem Solving: Effects on Learning“, *Cognitive Science*, Bd. 12, Nr. 2, S. 257–285, 1988, doi: 10.1207/s15516709cog1202_4.

[29] Microsoft, „User Interface | Visual Studio Code“. Zugegriffen: 12. März 2026. [Online]. Verfügbar unter: https://code.visualstudio.com/docs/getstarted/userinterface#_outline-view

[30] Microsoft, „Code Navigation | Visual Studio Code“. Zugegriffen: 12. März 2026. [Online]. Verfügbar unter: https://code.visualstudio.com/docs/editing/editingevolved#_go-to-definition

[31] SAP, „cap/cds-lsp: Language Server for CDS“. Zugegriffen: 12. März 2026. [Online]. Verfügbar unter: https://github.tools.sap/cap/cds-lsp

[32] A. J. Ko, R. DeLine, und G. Venolia, „Information Needs in Collocated Software Development Teams“, in *Proceedings of the 29th International Conference on Software Engineering (ICSE '07)*, IEEE Computer Society, Mai 2007, S. 344–353. doi: 10.1109/ICSE.2007.45.

[33] U. Götze, *Investitionsrechnung: Modelle und Analysen zur Beurteilung von Investitionsvorhaben*, 7. Aufl. Berlin, Heidelberg: Springer, 2014. doi: 10.1007/978-3-642-54622-8.

Anhang

Kriterium	Strategie	Punkte	Begründung
K1: Semantische Genauigkeit	Repository	2	Eigenes Parsing ohne Nutzung des CAP-Compilers; Risiko semantischer Fehlinterpretationen der CDS-Sprache
	CSN	5	Nutzt kompiliertes CAP-Modell; semantisch korrekte Interpretation durch den CAP-Compiler garantiert
	Language Server	5	Nutzt CAP-Compiler; semantische Analyse durch SAP-Tooling bereitgestellt
K2: Positionsinformationen	Repository	3	Positionen aus eigenem Parsing grundsätzlich verfügbar, aber aufwändige Implementierung erforderlich
	CSN	1	CSN enthält konstruktionsbedingt keine Quellcode-Positionen (Zeile/Spalte)
	Language Server	5	Location-API liefert präzise Positionen für Definitionen und Referenzen
K3: VS Code Integration	Repository	2	Keine native Integration; eigene Adapter erforderlich
	CSN	3	JSON-Daten müssen in VS-Code-Strukturen transformiert werden
	Language Server	5	Native LSP-Schnittstellen (DocumentSymbol, Location) direkt nutzbar
K4: Implementierungsaufwand	Repository	2	Hoher Aufwand: eigener Parser für CDS und JS/TS erforderlich
	CSN	4	Geringer Aufwand: JSON-Parsing und Traversierung des Modells
	Language Server	3	Mittlerer Aufwand: LSP-Client-Integration und Request-Handling
K5: Wartbarkeit	Repository	2	Eigener Parser muss bei CDS-Sprachänderungen angepasst werden
	CSN	3	CSN-Schema kann sich bei CAP-Updates ändern
	Language Server	4	LSP ist ein stabiler Standard; Änderungen werden vom LS abstrahiert
K6: Unabhängigkeit	Repository	5	Keine externen Abhängigkeiten zur Laufzeit
	CSN	2	Abhängigkeit vom CAP-Compiler zur Modellgenerierung
	Language Server	2	Abhängigkeit vom laufenden CAP Language Server

Tabelle 4 — Detaillierte Begründung der Punktvergabe in der Nutzwertanalyse

#	Datei	Zeile	Aktion
1	`srv/cat-service.cds`	1	Import-Statement und Namespace identifizieren
2	`srv/cat-service.cds`	3	Service-Definition `CatalogService` analysieren
3	`srv/cat-service.cds`	6–9	Projection `ListOfBooks` verstehen
4	`srv/cat-service.cds`	12–16	Projection `Books` verstehen
5	`srv/cat-service.cds`	19	Action `submitOrder` identifizieren
6	`db/schema.cds`	4–13	Entity `Books` mit Associations verstehen
7	`db/schema.cds`	15–23	Referenzierte Entity `Authors` analysieren
8	`db/schema.cds`	26–29	Referenzierte Entity `Genres` analysieren
9	`srv/cat-service.js`	3–6	Handler-Klasse und Entity-Referenzen
10	`srv/cat-service.js`	9–23	Event Handler (`after`, `on`) analysieren

Tabelle 5 — Szenario „Service verstehen“ – 3 Dateien, 10 Navigationsschritte

#	Datei	Zeile	Aktion
1	`db/schema.cds`	4–13	Entity `Books` finden, Feld hinzufügen
2	`srv/cat-service.cds`	6–16	Projektionen `ListOfBooks` und `Books` anpassen
3	`srv/admin-service.cds`	5	AdminService Projection prüfen
4	`srv/admin-constraints.cds`	4–18	Validierungs-Constraints prüfen
5	`srv/cat-service.js`	9–23	CatalogService Handler prüfen
6	`srv/admin-service.js`	4–5	AdminService Handler prüfen

Tabelle 6 — Szenario „Feld hinzufügen“ – 6 Dateien, 6 Navigationsschritte

#	Datei	Zeile	Aktion
1	`srv/cat-service.cds`	3	Service-Definition identifizieren
2	`srv/cat-service.cds`	18–19	`@requires`-Annotation und betroffene Action
3	`db/schema.cds`	4–29	Exponierte Daten auf Sensibilität prüfen
4	`srv/cat-service.js`	14–23	Handler auf Auth-abhängige Logik prüfen

Tabelle 7 — Szenario „Auth anpassen“ – 3 Dateien, 4 Navigationsschritte

#	Datei	Zeile	Aktion
1	`srv/cat-service.js`	14	`submitOrder`-Handler finden
2	`srv/cat-service.js`	15–22	Handler-Logik analysieren
3	`srv/cat-service.cds`	19	Action-Definition und Parameter prüfen
4	`db/schema.cds`	4–13	Entity `Books` mit `stock`-Feld verstehen

Tabelle 8 — Szenario „Handler debuggen“ – 3 Dateien, 4 Navigationsschritte